

LumaPath — Project Flaw Report

Full review of the backend (FastAPI + ML) and frontend (React + Leaflet). Prepared 24 September 2026 against commit `888cbb5` plus the staged ML changes. Every flaw has an ID, a severity, the file and line where it lives, the problem, and how to fix it.

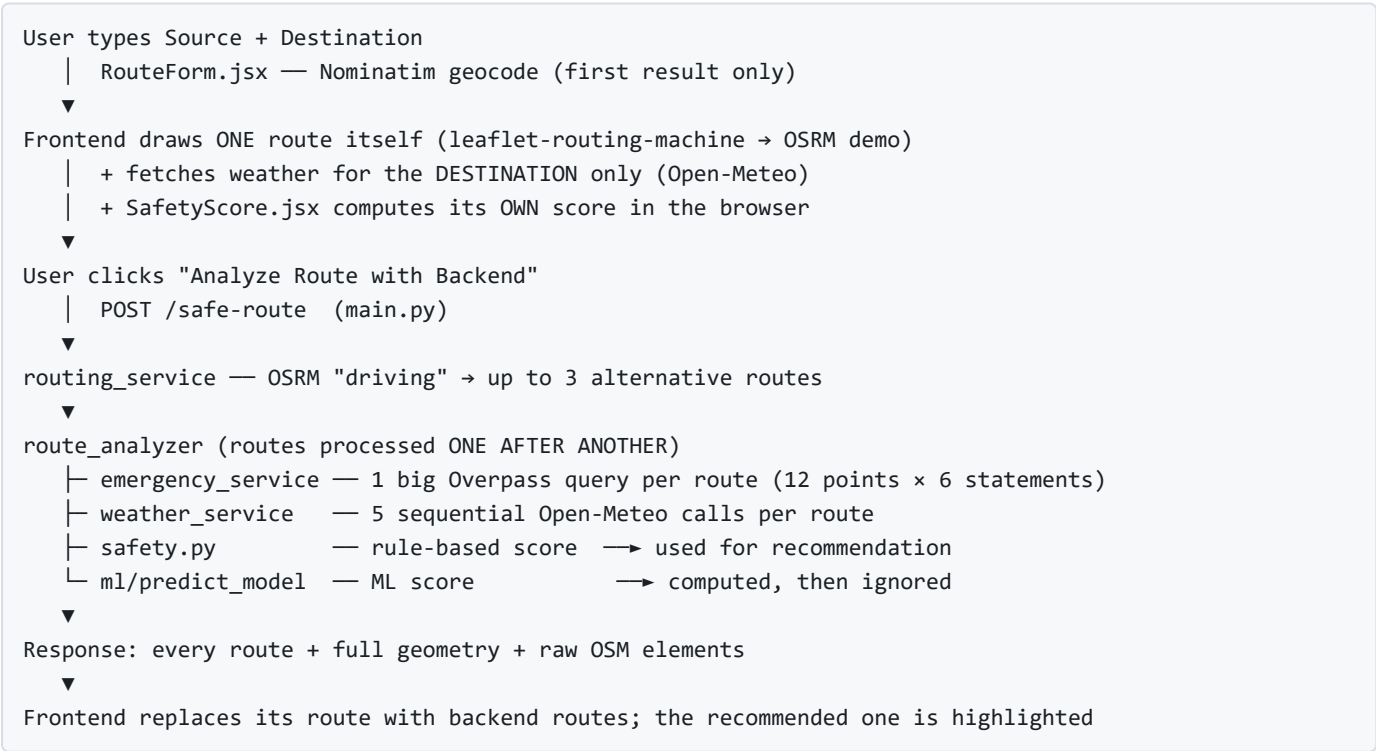
1. Summary

Area	Critical	High	Medium	Low	Total
A. Safety logic & ML (core of the product)	3	4	1	0	8
B. Functional bugs	1	3	5	1	10
C. Performance & scalability	0	2	3	0	5
D. Security & privacy	0	3	3	1	7
E. UX & accessibility	0	1	3	1	5
F. Code quality & engineering	0	1	5	3	9
Total	4	14	20	6	44

Severity key. CRITICAL gives users wrong safety information or breaks the main flow. HIGH is a serious bug, risk or blocker for deployment. MEDIUM should be fixed before a real release. LOW is cleanup or polish.

What is already good. The secret `backend/.env` is gitignored and has never been committed. The OSRM lon/lat ordering is handled correctly on both sides. Overpass has a fallback server. Alternative routes and the route-comparison UI work end to end.

2. How the app flows today



Main structural problems in this flow: there are two routing engines and two safety scores that can disagree (A1, B1). The whole flow runs sequentially, so it is slow (C1). A new search does not clear the old results (B2).

3. A — Safety logic & ML

CRITICAL A1. Two different safety scores are shown, and they can disagree

frontend/src/components/SafetyScore.jsx:1-48 vs backend/safety.py:1-45

The browser computes its own score. It uses different thresholds (rain ≥ 5 mm = -20 vs backend > 10 mm = -15), uses weather at the *destination only*, and pools hospitals and police from *all* routes combined. The backend computes a different score for each route. A user can see "Route Safety Score 80/100 — GOOD" next to "Recommended Route — Safety Score 65, Moderate Risk".

Fix: Treat the backend as the single source of truth. Delete the scoring logic in `SafetyScore.jsx` and render `safety_score`, `risk_level` and a list of reasons returned by the backend for the *selected* route. Have `calculate_safety_score` also return `warnings: [...]` so the UI can explain the score.

CRITICAL A2. The score does not measure what makes a route unsafe for the target users

backend/safety.py:1-45, backend/ml/train_model.py:48-55

The README says the target users are women, elderly people and children. The only inputs are the number of hospitals and police stations and the current rain and wind. The score ignores the factors that matter most for personal safety: street lighting, time of day (night vs day), isolated stretches, road type (highway vs residential), footpaths, crowd and activity levels, and crime history.

Fix:

- Get these from OSM through the Overpass query you already run: `highway=street_lamp`, `lit=yes/no` on ways, `highway=* type`, `sidewalk=*`, and shops, bus stops and 24h amenities as a proxy for "active streets".
- Add a time-of-day factor. Pass the departure time from the frontend and apply a stronger penalty for unlit or isolated segments at night.
- Add public crime or incident data where it is available (for example state police open data), plus in-app user reports ("felt unsafe here").
- Score *segments* rather than whole routes, and report the worst segment. One dark 2 km stretch matters more than the average.

CRITICAL A3. Failed API calls silently count as "perfect conditions"

backend/weather_service.py:89-106, backend/emergency_service.py:188-198

If Open-Meteo fails, weather comes back as `0` precipitation and `0` wind, so the route looks like it has perfect weather. If both Overpass servers fail, every route gets 0 hospitals and 0 police. The user then sees "Moderate Risk" that comes from missing data, not from reality, and nothing tells them the data is missing.

Fix: Return `None` or `{"available": False}` instead of zeros. Skip the unavailable factor when scoring, and add a `data_quality` / `warnings` field ("Weather data unavailable — score based on partial information"). Show that clearly in the UI.

HIGH A4. The score favours longer routes

backend/emergency_service.py:18-37, backend/route_analyzer.py:115-121, backend/safety.py:11-19

The code counts hospitals and police within 1.5 km of 12 sampled points. A 40 km detour sweeps far more area than a 5 km direct route, so it collects more hospitals and police and scores "safer". The samples are also uneven: 12 points on 5 km is one every 420 m, but 12 points on 50 km is one every 4.5 km, which leaves big unchecked gaps. The thresholds (`police < 2`, `hospitals < 3`) are arbitrary absolute counts.

Fix: Sample by distance (for example every 300–500 m), not by a fixed count. Replace raw counts with *coverage* metrics: the percentage of the route within X km of a police station or hospital, and the *longest gap* with no service nearby. Normalise per km.

HIGH A5. Routes are computed for cars, but the users are pedestrians

backend/routing_service.py:8 (`/route/v1/driving`); MapView.jsx:151 (leaflet-routing-machine defaults to driving)

Driving routes use highways, flyovers and one-way logic. They are wrong for someone walking, and the durations shown are driving times.

Fix: Add a travel-mode selector (Walk / Two-wheeler / Car). For walking use a foot profile, for example `https://routing.openstreetmap.de/routed-foot/route/v1/foot/...`, `OpenRouteService` `foot-walking`, `GraphHopper`, or a self-hosted OSRM with `foot.lua`. The public `router.project-osrm.org` server supports only the car profile.

HIGH A6. The ML model is trained on 10 invented rows, and its output is thrown away

backend/ml/dataset.csv (10 rows), train_model.py:72-77, route_analyzer.py:264-284, main.py:262-280

- The dataset has 10 hand-written rows. After the 80/20 split, the model trains on 8 rows and tests on 2, so the MAE and R^2 it prints mean nothing.
- In every row, longer distance goes with a lower score, so the model mostly learns "short = safe". It is not a safety signal.
- A Random Forest cannot extrapolate. Any input outside the 10 examples gets clamped to the nearest leaf values.
- `ml_safety_score` is returned but never used: not in the recommendation (`main.py` uses the rule score) and not in the UI.

Fix: Keep the rule-based score for now and be honest in the UI that it is rule-based. Label ML as experimental until you have real labelled data (user ratings of routes, incident reports, crowd-sourced "felt safe" feedback). When you have data: use k-fold cross-validation, keep a held-out test set, compare against the rule-based baseline, and save the model with a version number and its metrics. Remove `distance_km / duration_min` as safety features, or justify them.

HIGH A7. The recommendation ignores detour cost

backend/main.py:262-280; unused helper backend/safety.py:72-113

The route with the highest safety score always wins. Distance only breaks ties. A 35 km route scoring 100 beats a 6 km route scoring 95. `calculate_recommendation_score`, which balances safety against distance, already exists but is never called.

Fix: Use `recommend_best_route` (tune its weights), or recommend the shortest route within N points of the best score. Show the trade-off (" +4 min, noticeably safer").

MEDIUM A8. Weather is "right now", not when the user will be on the road

backend/weather_service.py:30-35 (`current=`), MapView.jsx:252-259

The code uses current conditions only. The frontend shows weather for the destination point only, even for long routes.

Fix: Use Open-Meteo `hourly` forecast values for the estimated time at each point on the route. Show the worst condition along the route, not only the destination's.

4. B — Functional bugs

CRITICAL B1. Two routing engines draw different routes

frontend/src/components/MapView.jsx:118-331 (leaflet-routing-machine) vs backend/routing_service.py

Before analysis, the map shows the frontend's own route, and the distance and time cards come from it. After analysis, the backend routes replace it, but `RouteInfo` still shows the *frontend* distance and time, which do not match the selected backend route.

Fix: Remove leaflet-routing-machine. Call `/safe-route` directly when the user clicks "Find Safe Route", and drive every card (distance, time, weather, score) from the selected backend route.

HIGH B2. Old results stay on screen after a new search

frontend/src/pages/Home.jsx:134-140 (reset only inside `handleSafeRoute`); MapView.jsx:1031-1035

When the user searches a new source and destination, `safeRouteData`, `selectedRoute`, `distance`, `time` and `weather` are not cleared. Because `Routing` renders only when `!safeRouteData`, the map keeps showing the **previous trip's routes, hospitals and recommendation** next to the new markers.

Fix: Add a `resetResults()` in Home and call it whenever new coordinates are set, for example in a `useEffect` on `[sourceCoords, destinationCoords]` or directly in RouteForm's submit handler.

HIGH B3. Import path case mismatch breaks builds on Linux and macOS

frontend/src/components/RouteForm.jsx:1 — `"../services/geocodingService"`; the folder is `Services`

This works on Windows, whose filesystem ignores case. It fails on Linux, macOS, CI and hosting services such as Vercel or Netlify with "module not found".

Fix: Change it to `"../Services/geocodingService"`, or better, rename the folder to lowercase `services` and update every import.

HIGH B4. The backend only starts from inside the `backend/` folder, and crashes if the model is missing

backend/main.py:5-6, route_analyzer.py:1-15, ml/predict_model.py:24-36

The imports are top-level (`from routing_service import ...`, `from ml.predict_model import ...`), so `uvicorn backend.main:app` from the repo root fails. `predict_model` raises `FileNotFoundError` at import time, which takes down the whole API even though the ML score is unused.

Fix: Make `backend` a package (`__init__.py`, relative imports) or document the run command. Load the model lazily and fall back to `None` with a logged warning.

MEDIUM B5. Geocoding takes the first match and can leave half-updated state

frontend/src/components/RouteForm.jsx:74-98, Services/geocodingService.js

- It uses only the first Nominatim result, with no country or region bias. Ambiguous names ("Kottayam", "Model School") can resolve to the wrong place or even another country.
- If the source resolves but the destination fails, the source coordinates are already updated, which leaves inconsistent state.
- The string `"Current Location"` is used as a flag. If the user types that text, stale coordinates are used.

Fix: Resolve both first and set both only if both succeed. Add `countrycodes=in` and a `viewbox`. Show an autocomplete or "did you mean" list. Track `useCurrentLocation` as a boolean state, not a magic string.

MEDIUM B6. Default Leaflet markers probably show as broken images

frontend/src/components/MapView.jsx:980-1019 (source and destination use the default icon)

This is a known problem with Leaflet under Vite and Webpack: `L.Icon.Default` resolves its image paths incorrectly once bundled, so the markers show as broken images or nothing.

Fix: Import `marker-icon.png`, `marker-icon-2x.png` and `marker-shadow.png` from `leaflet/dist/images` and call `L.Icon.Default.mergeOptions({...})` once, or use custom icons for source and destination.

MEDIUM B7. RouteComparison crashes if there is no recommended route

frontend/src/components/RouteComparison.jsx:18

`safeRouteData.recommended_route.name` has no optional chaining, so a missing or null `recommended_route` throws and blanks the page.

Fix: Use `safeRouteData.recommended_route?.name` and guard `safeRouteData.routes`.

MEDIUM B8. Broken and empty files

frontend/src/Layout.jsx:1 (imports `./Navbar`, which does not exist); pages/Login.jsx and pages/LoginForm.jsx (0 bytes)

Fix: Fix Layout's import to `./components/Navbar` and use Layout in App.jsx so each page stops rendering its own Navbar. Delete the empty Login files or implement them.

MEDIUM B9. ESLint fails with 4 errors

Home.jsx:354 (setState in effect), SafetyScore.jsx:40, Counter.jsx:4, overpassService.js:67

Fix: Home.jsx: compute hospitals and police with `useMemo` from `safeRouteData` instead of copying them into state in an effect. Remove the dead code (see F3). Run `npm run lint` in CI.

LOW B10. The database layer is half-built and runs at import time

backend/database.py:42-49, backend/models.py

`database.py` opens a DB connection and prints the result as soon as it is imported. `models.py` defines `Route`, but nothing creates the tables or uses them, and `main.py` never imports either file.

Fix: Either wire it up (save search history and user reports, and use Alembic migrations) or remove it until you need it. Move the connection test into a `/health` check.

5. C — Performance & scalability

HIGH C1. Every external call runs in sequence, so one request can take minutes

backend/route_analyzer.py:384-395 (routes in a loop), 168-198 (weather in a loop); emergency_service.py:127-181

For 3 routes the backend makes 3 Overpass requests (60 s query timeout, 75 s client timeout, 2 servers) plus 15 weather calls, **all one after another**. The worst case is well over 5 minutes, and a typical request takes 10–40 s.

Fix:

- Run the routes concurrently with `asyncio.gather(*(analyze_route(r) for r in routes))`.
- Replace the 5 weather calls with **one** Open-Meteo call. It accepts comma-separated lists: `latitude=10.1,10.2,...&longitude=76.1,76.2,...`.
- Make **one** Overpass request for all routes together, then assign each POI to the routes it is near using a distance check in Python.
- Share one `httpx.AsyncClient` (create it in the FastAPI lifespan) instead of opening a new one for each call.

HIGH C2. No caching, so repeated searches hit public servers every time

all backend services

Hospitals and police stations barely change, and weather changes hourly. The same query repeated 10 times makes about 60 external calls.

Fix: Add a TTL cache keyed on rounded coordinates: POIs for 24 h, weather for 15–30 min, routes for 1 h. Use `cachetools.TTLCache` in memory first, and Redis later.

MEDIUM C3. The Overpass query is bigger than it needs to be

backend/emergency_service.py:80-117

It sends 72 `around` statements (12 points × node/way/relation × 2 amenities) per route, and the circles overlap heavily.

Fix: Overpass supports a **polyline** around filter, `(around:1500,lat1,lon1,lat2,lon2,...)`, which covers the whole route. Combine that with `nwr["amenity"~"^(hospital|police)$"]` for a single statement.

MEDIUM C4. The response payload is very large

backend/route_analyzer.py:304-370 (`**route` + raw OSM elements)

Every route returns its full-resolution GeoJSON geometry plus raw OSM elements with every tag. The same hospital is repeated in several routes.

Fix: Return slim POIs (`id`, `name`, `lat`, `lon`, `type`), deduplicate them once at the top level, and reference them by id from each route. Simplify or encode the geometry (Douglas-Peucker or `polyline6`). Define a Pydantic `response_model`.

MEDIUM C5. The same data is processed twice in the frontend

Home.jsx:225-366 and MapView.jsx:455-556 (identical dedup logic)

Fix: Deduplicate once, ideally in the backend (C4), and pass the result down as props.

6. D — Security & privacy

HIGH D1. User locations are logged

backend/main.py:115-125; frontend Home.jsx:148-157, RouteForm.jsx:25-27, MapView.jsx:262

The exact source and destination coordinates, meaning where a woman, child or elderly person lives and where they are going, are printed to server logs and the browser console. For this audience that is sensitive personal data.

Fix: Remove these prints. Use Python `logging` at `DEBUG` level with rounded coordinates if you need them. Strip `console.log` from production builds (`esbuild.drop: ['console']` in the Vite config). Add a short privacy notice.

HIGH D2. No rate limiting, while the app depends on public APIs with strict usage policies

backend/main.py:97; routing_service.py:8; emergency_service.py:8-11; geocodingService.js:3

Each `/safe-route` call triggers about 20 third-party requests. The OSRM demo server, Overpass and Nominatim are all "fair use, not for production", and Nominatim allows 1 request per second with an identifying User-Agent. A small traffic spike can get your server IP banned, and then the app stops working.

Fix: Add per-IP limits with `slowapi`. Add caching (C2). Move geocoding to the backend with a proper User-Agent and contact email. For production, self-host OSRM and Nominatim or use a keyed provider (OpenRouteService, GraphHopper, LocationIQ).

HIGH D3. Internal errors are sent to the client

backend/main.py:153-165, 224-236 (`"error": str(error)`)

Exception text (upstream URLs, stack details) is returned to the browser.

Fix: Log the exception server-side with `logger.exception` and return a generic message plus a request id. Use `502` for upstream failures instead of `500`.

MEDIUM D4. No input validation

backend/main.py:57-65

Latitude and longitude accept any float (for example 999 or NaN), and there is no limit on trip length, so a cross-continent request makes a huge Overpass query.

Fix: `source_lat: float = Field(ge=-90, le=90)`, `source_lon: float = Field(ge=-180, le=180)`, and reject trips over a sensible maximum (for example 100 km straight-line).

MEDIUM D5. The pickled model is loaded blindly, and a duplicate copy sits in the repo root

backend/ml/predict_model.py:34; stray `/safety_model.pkl` and empty `/ml` file in the repo root (both staged)

`joblib.load` runs arbitrary code if someone replaces the file. The root copy is a leftover that should not be committed.

Fix: Unstage and delete `/safety_model.pkl` and `/ml`. Verify a SHA-256 checksum before loading, or export to a non-executable format (skops or ONNX). Consider Git LFS or regenerating the model in CI instead of committing binaries.

MEDIUM D6. Loose CORS and hard-coded origins

backend/main.py:31-50

`allow_credentials=True` together with `allow_methods=["*"]` and `allow_headers=["*"]` is unnecessary because the app uses no cookies or auth. The origins are hard-coded to localhost.

Fix: Read origins from an env var, set `allow_credentials=False`, and allow only GET and POST with the Content-Type header.

LOW D7. Marker icons are hot-linked from third-party hosts

frontend/src/components/MapView.jsx:25-56 (raw.githubusercontent.com, unpkg.com)

These hosts can see your users' IP addresses, and the icons disappear if a host changes or rate-limits the files.

Fix: Copy the icons into `src/assets` and import them.

7. E — UX & accessibility (key for elderly users)

HIGH E1. A confusing two-step flow that exposes internal terms

frontend/src/pages/Home.jsx:522-562

The user clicks "Find Safe Route", sees a route that has not been analysed, and must then find a second button labelled "Analyze Route with *Backend*". Many users will stop after step one and trust an unscored route.

Fix: Use one button that geocodes, calls `/safe-route`, shows a progress state ("Checking hospitals... weather...") and then shows the results. Never show the word "backend" to users.

MEDIUM E2. Placeholder pages and dead buttons

Hero.jsx:16 ("Get Started" has no handler); pages/MapPage.jsx, About.jsx, Emergency.jsx are a heading only; index.html title is "frontend"

Fix: Make "Get Started" scroll to the route form. Either build the pages or hide them from the Navbar. Set the page title to "LumaPath — Safe Route Navigation".

MEDIUM E3. Not accessible and not mobile-friendly

RouteComparison.jsx:21-42 (clickable `div`); RouteForm.css (`width: 400px`); RouteInfo.jsx (`width: 300px`); MapView (red/blue/green/gray only)

- Route cards cannot be reached with the keyboard or a screen reader.
- Selection and risk are shown by colour only, which colour-blind users cannot distinguish.
- Fixed pixel widths overflow on phones, where this app will mostly be used.
- Errors use blocking `alert()` popups.

Fix: Use `<button>` or `role="radio"` for route cards. Add text and icon labels to colours. Use `max-width: 100%` and responsive CSS with an 18px+ base font. Show errors inline or as toasts with plain-language messages.

MEDIUM E4. The SOS feature is minimal for a safety app

frontend/src/components/EmergencySOS.jsx

It offers only `tel:` links. It does not share the user's location, has no trusted contacts, and does not check in during the trip.

Fix: Add an "SOS: share my live location" action that opens SMS or WhatsApp with a Google Maps or OSM link to a saved contact. Add a "share trip" link and an optional "I reached safely" timer. Also add 1091 (women's helpline) and 1098 (child helpline), which match the target users.

LOW E5. No loading feedback while geocoding and routing

RouteForm.jsx:66-103, RouteStatus.jsx

Fix: Disable the button and show a spinner while requests are in flight. `RouteStatus` should show an error state as well as "Finding Route...".

8. F — Code quality & engineering

HIGH F1. No dependency list for the backend

backend/ (no requirements.txt or pyproject.toml)

Nobody else, including a deployment server, can recreate the venv. The packages are fastapi, uvicorn, httpx, sqlalchemy, python-dotenv, pandas, scikit-learn and joblib.

Fix: `pip freeze > backend/requirements.txt` (or use a pyproject with pinned versions), and add setup and run steps to the README.

MEDIUM F2. No automated tests

Fix: Backend: `pytest` for `calculate_safety_score`, point sampling and the recommendation choice, with `respx` to mock `httpx` so tests do not call real APIs. Also add a `TestClient` test for `/safe-route`. Frontend: `Vitest` and `React Testing Library` for `RouteForm` and `RouteComparison`. Run both in GitHub Actions.

MEDIUM F3. Duplicate and dead code

`get_route_points` (`emergency_service.py:18`) duplicates `sample_route_points` (`route_analyzer.py:22`); unused: `safety.compare_routes`, `safety.recommend_best_route`, `Services/overpassService.js`, `Services/weatherService.js` (after B1), components `Counter`, `NameInput`, `Button`

Fix: Keep one `sample_points()` in a `geo_utils.py`. Delete the unused files, which were practice components.

MEDIUM F4. Configuration is hard-coded

frontend/src/Services/api.js:3 (`http://127.0.0.1:8000`); URLs, radius (1500), sample counts and score thresholds throughout the backend

Fix: Frontend: `import.meta.env.VITE_API_URL`. Backend: one `config.py` using `pydantic-settings` that reads from `.env`. Commit a `.env.example`.

MEDIUM F5. `print()` is used for logging, and broad `except Exception` blocks hide failures

every backend module

Fix: Use `logging.getLogger(__name__)` with levels. Catch specific errors (`httpx.HTTPError`, `httpx.TimeoutException`, `ValueError`) and let the rest propagate to one exception handler.

MEDIUM F6. No API contract

backend/main.py:332-345

The API returns plain dicts, so the response shape can drift and break the frontend without anyone noticing.

Fix: Define Pydantic `RouteResult` and `SafeRouteResponse` models and set `response_model=`. Version the path (`/api/v1/safe-route`).

LOW F7. The formatting makes the files 3–4× longer than they need to be

e.g. `MapView.jsx` (1096 lines), `Home.jsx` (748), `route_analyzer.py` (406)

Splitting each argument across three lines with blank lines between them makes the code hard to read and review.

Fix: Run `black` or `ruff` format on Python and `prettier` on JS, and add a pre-commit hook. Split `MapView` into `RouteLayer`, `PoiMarkers` and `icons.js`.

LOW F8. Inline styles everywhere

`EmergencySOS`, `Footer`, `MapLegend`, `RouteInfo`, `RouteStatus`, `SafetyScore`, `WeatherInfo`, `Home`

Fix: Move the styles into CSS files or CSS modules with shared variables for colours and spacing, so the look stays consistent and can be themed (for example high contrast).

LOW F9. Documentation is minimal

`README.md` (2 lines), `frontend/README.md` (Vite template)

Fix: Document the architecture (the diagram in section 2), setup for both halves, environment variables, how the safety score works and its limitations, and the external APIs and their usage limits.

9. Suggested order of work

Phase	Items	Why first
1. Quick fixes (1–2 days)	B2, B3, B7, B8, B9, D3, D4, D5 (remove stray files), F1	Small changes that remove crashes, build failures and the stale-results bug.
2. One flow, one score	A1, B1, E1, A3, A7, B5	The user sees one route source and one explainable score, and failures are visible.
3. Speed & reliability	C1, C2, C3, C4, D2, B4	Responses go from tens of seconds to about 2–5 s, and the app is protected from API bans.
4. Real safety	A5 (walking mode), A4, A2, A8, E4	Makes "safe route" mean what the target users need.
5. ML done properly	A6	Only after real labelled data exists.

6. Hygiene

D1, D6, D7, E2, E3, E5, F2–F9, B6,
B10

Maintainability, privacy, polish.